

LangChain Expression Language (LCEL)

```
In [1]: import os
import openai

from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file
openai.api_key = os.environ['OPENAI_API_KEY']
```

```
In [2]: #!pip install pydantic==1.10.8
```

```
In [3]: from langchain.prompts import ChatPromptTemplate
from langchain.chat_models import ChatOpenAI
from langchain.schema.output_parser import StrOutputParser
```

Simple Chain

```
In [4]: prompt = ChatPromptTemplate.from_template(
    "tell me a short joke about {topic}"
)
model = ChatOpenAI()
output_parser = StrOutputParser()
```

```
In [5]: chain = prompt | model | output_parser
```

```
In [6]: chain.invoke({"topic": "bears"})
```

```
Out[6]: 'Why did the bear bring a flashlight to the party? \n\nBecause he heard it
was going to be a "bear"y bright night!'
```

More complex chain

And Runnable Map to supply user-provided inputs to the prompt.

```
In [7]: from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import DocArrayInMemorySearch
```

```
In [8]: vectorstore = DocArrayInMemorySearch.from_texts(
    ["harrison worked at kensho", "bears like to eat honey"],
    embedding=OpenAIEmbeddings()
)
retriever = vectorstore.as_retriever()
```

```
In [9]: retriever.get_relevant_documents("where did harrison work?")
```

```
Out[9]: [Document(page_content='harrison worked at kensho'),
Document(page_content='bears like to eat honey')]
```

```
In [10]: retriever.get_relevant_documents("what do bears like to eat")
```

```
Out[10]: [Document(page_content='bears like to eat honey'),  
          Document(page_content='harrison worked at kensho')]
```

```
In [11]: template = """Answer the question based only on the following context:  
          {context}  
  
          Question: {question}  
          """  
          prompt = ChatPromptTemplate.from_template(template)
```

```
In [12]: from langchain.schema.runnable import RunnableMap
```

```
In [14]: RunnableMap({  
          "context": lambda x: retriever.get_relevant_documents(x["question"]),  
          "question": lambda x: x["question"]  
          })
```

```
Out[14]: {  
          context: RunnableLambda(...),  
          question: RunnableLambda(...)  
          }
```

```
In [15]: chain = RunnableMap({  
          "context": lambda x: retriever.get_relevant_documents(x["question"]),  
          "question": lambda x: x["question"]  
          }) | prompt | model | output_parser
```

```
In [16]: chain.invoke({"question": "where did harrison work?"})
```

```
Out[16]: 'Harrison worked at Kensho.'
```

```
In [17]: inputs = RunnableMap({  
          "context": lambda x: retriever.get_relevant_documents(x["question"]),  
          "question": lambda x: x["question"]  
          })
```

```
In [18]: inputs.invoke({"question": "where did harrison work?"})
```

```
Out[18]: {'context': [Document(page_content='harrison worked at kensho'),  
                      Document(page_content='bears like to eat honey')],  
          'question': 'where did harrison work?'}
```

Bind

and OpenAI Functions

```
In [19]: functions = [  
          {  
              "name": "weather_search",  
              "description": "Search for weather given an airport code",
```

```

        "parameters": {
            "type": "object",
            "properties": {
                "airport_code": {
                    "type": "string",
                    "description": "The airport code to get the weather for"
                },
            },
            "required": ["airport_code"]
        }
    }
]

```

In [20]: *# using the 'bind' - modifying parameters at runtime*

```

prompt = ChatPromptTemplate.from_messages(
    [
        ("human", "{input}")
    ]
)
model = ChatOpenAI(temperature=0).bind(functions=functions)

```

In [21]: `runnable = prompt | model`

In [22]: `runnable.invoke({"input": "what is the weather in sf"})`

Out[22]: `AIMessage(content='', additional_kwargs={'function_call': {'name': 'weather_search', 'arguments': '{"airport_code": "SF0"}'}})`

In [23]:

```

functions = [
    {
        "name": "weather_search",
        "description": "Search for weather given an airport code",
        "parameters": {
            "type": "object",
            "properties": {
                "airport_code": {
                    "type": "string",
                    "description": "The airport code to get the weather for"
                },
            },
            "required": ["airport_code"]
        }
    },
    {
        "name": "sports_search",
        "description": "Search for news of recent sport events",
        "parameters": {
            "type": "object",
            "properties": {
                "team_name": {
                    "type": "string",
                    "description": "The sports team to search for"
                },
            },
        },
    },
]

```

```

        "required": ["team_name"]
    }
}
]

```

```
In [24]: model = model.bind(functions=functions)
```

```
In [25]: runnable = prompt | model
```

```
In [26]: runnable.invoke({"input": "how did the patriots do yesterday?"})
```

```
Out[26]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'sports_search', 'arguments': '{"team_name": "New England Patriots"}'}})
```

Fallbacks

```
In [27]: from langchain.llms import OpenAI
import json
```

Note: Due to the deprecation of OpenAI's model `text-davinci-001` on 4 January 2024, you'll be using OpenAI's recommended replacement model `gpt-3.5-turbo-instruct` instead.

```
In [28]: simple_model = OpenAI(
    temperature=0,
    max_tokens=1000,
    model="gpt-3.5-turbo-instruct"
)
simple_chain = simple_model | json.loads
```

```
In [29]: challenge = "write three poems in a json blob, where each poem is a json blob"
```

```
In [30]: simple_model.invoke(challenge)
```

```
Out[30]: '\n\n{\n  "title": "Autumn Leaves",\n  "author": "Emily Dickinson",\n  "first_line": "The leaves are falling, one by one"\n}\n\n{\n  "title": "The Ocean\'s Song",\n  "author": "Pablo Neruda",\n  "first_line": "I hear the ocean\'s song, a symphony of waves"\n}\n\n{\n  "title": "A Winter \'s Night",\n  "author": "Robert Frost",\n  "first_line": "The snow falls softly, covering the ground"\n}'
```

Note: The next line is expected to fail.

```
In [ ]: simple_chain.invoke(challenge)
```

```
In [32]: model = ChatOpenAI(temperature=0)
chain = model | StrOutputParser() | json.loads
```

```
In [33]: chain.invoke(challenge)
```

```
Out[33]: {'poem1': {'title': 'The Rose',
  'author': 'Emily Dickinson',
  'firstLine': 'A rose by any other name would smell as sweet'},
  'poem2': {'title': 'The Road Not Taken',
  'author': 'Robert Frost',
  'firstLine': 'Two roads diverged in a yellow wood'},
  'poem3': {'title': 'Hope is the Thing with Feathers',
  'author': 'Emily Dickinson',
  'firstLine': 'Hope is the thing with feathers that perches in the soul'}}
```

```
In [34]: final_chain = simple_chain.with_fallbacks([chain])
```

```
In [35]: final_chain.invoke(challenge)
```

```
Out[35]: {'poem1': {'title': 'The Rose',
  'author': 'Emily Dickinson',
  'firstLine': 'A rose is a rose is a rose'},
  'poem2': {'title': 'The Road Not Taken',
  'author': 'Robert Frost',
  'firstLine': 'Two roads diverged in a yellow wood'},
  'poem3': {'title': 'Hope is the Thing with Feathers',
  'author': 'Emily Dickinson',
  'firstLine': 'Hope is the thing with feathers'}}
```

Interface

```
In [36]: prompt = ChatPromptTemplate.from_template(
  "Tell me a short joke about {topic}"
)
model = ChatOpenAI()
output_parser = StrOutputParser()

chain = prompt | model | output_parser
```

```
In [37]: chain.invoke({"topic": "bears"})
```

```
Out[37]: "Why did the bear break up with his girlfriend? \n\nBecause he couldn't bear the relationship anymore!"
```

```
In [38]: chain.batch([{"topic": "bears"}, {"topic": "frogs"}])
```

```
Out[38]: ['Why did the bear bring a flashlight to the party? \nBecause it wanted to be the "light" of the party!',
  'Why did the frog take the bus to work? Because his car got toad away!']
```

```
In [ ]: for t in chain.stream({"topic": "bears"}):
  print(t)
```

```
In [ ]: response = await chain.ainvoke({"topic": "bears"})
response
```